

This architecture achieves a standard read-write OS drive while eliminating micro-writes and preventing file-transfer software from using the OS SSD as intermediate staging storage.

Step 1: Kernel Dirty-Page Tuning (RAM Batch Writing)

By adjusting the Linux Virtual Memory (VM) dirty-page subsystem, all write operations are retained in the 8GB RAM cache and flushed to the OS SSD in large sequential bursts only when the buffer threshold is reached or when the system initiates a shutdown/reboot.

1. Open /etc/sysctl.conf:

```
Bash
sudo nano /etc/sysctl.conf
```

2. Append the following parameters at the bottom:

```
Ini, TOML
# Hold dirty data in RAM until it reaches 20% of free memory (~1.6 GB)
vm.dirty_background_ratio = 20

# Force flush only if dirty data exceeds 40% of memory (~3.2 GB)
vm.dirty_ratio = 40

# Data can stay dirty in RAM for up to 30 minutes (180,000 centiseconds)
vm.dirty_expire_centisecs = 180000

# Wakeup interval for the kernel flush thread (every 30 minutes)
vm.dirty_writeback_centisecs = 180000
```

3. Apply changes:

```
Bash
sudo sysctl -p
```

4. Open /etc/fstab and set the filesystem commit interval and access-time suppression on the root (/) mount:

```
Plaintext
PARTUUID=xxxx-xx / ext4 noatime,commit=1800,errors=remount-ro 0 1
```

- commit=1800: Ext4 delays journal updates and syncs metadata to the SSD only once every 30 minutes.
- noatime: Disables write timestamps on read operations.

Step 2: Route System Logs and Temporary Dirs to RAM

Background Linux system logs and /tmp files cause thousands of 4KB random writes daily.

1. **Move /tmp and /var/tmp to RAM (tmpfs):**

Add to /etc/fstab:

```
Plaintext
tmpfs /tmp tmpfs defaults,noatime,nosuid,size=1G 0 0
tmpfs /var/tmp tmpfs defaults,noatime,nosuid,size=512M 0 0
```

2. Buffer /var/log with Log2RAM:

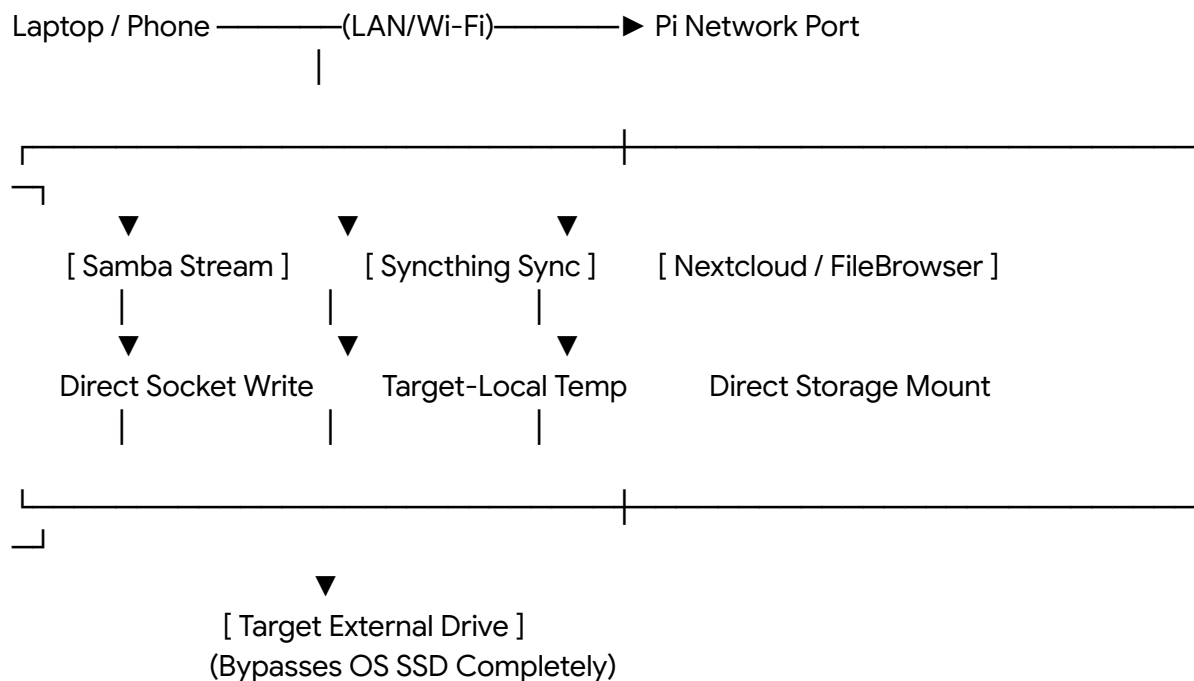
Bash

```
sudo apt install log2ram -y
```

Configure /etc/log2ram.conf with SIZE=256M. System logs will reside entirely in RAM and flush to disk once daily or upon clean shutdown.

Step 3: Zero-Buffer Direct-Path Transfer Setup

To ensure Nextcloud, Syncthing, Samba, and FileBrowser write directly from your network interface (LAN/Wi-Fi) to the destination drive without touching the OS SSD:



1. Nextcloud Direct Staging

By default, Nextcloud uploads file chunks to the OS /tmp before assembling them on the data drive.

- Open Nextcloud's config/config.php and set the upload chunk buffer directly to a temporary directory on the **target storage drive**:
PHP
'tempdirectory' => '/mnt/target_drive_1/nc_temp',
- Nextcloud will stream chunks over the network interface straight to the destination drive, executing zero writes on the OS SSD.

2. Syncthing Staging

- Syncthing defaults to writing .syncthing.filename.tmp in the **exact folder where the file belongs**.
- Ensure all folder paths in the Syncthing Web UI point strictly to /mnt/target_drive_X/... and not to any user home directory paths on the OS drive.

3. Samba (SMB) Direct Streaming

- Samba writes network socket streams directly into file handles on the target mount.
- In /etc/samba/smb.conf, verify that all shares point directly to the mounted disks and disable OS-level caching buffers:

Ini, TOML

[StorageDrive1]

path = /mnt/target_drive_1/shares

read only = no

aio read size = 1

aio write size = 1

strict sync = no

4. FileBrowser Direct Rooting

- Ensure FileBrowser's root directory is set directly to your storage mount path:
Bash
filebrowser config set --root=/mnt/
- Place the filebrowser.db database on an external drive or secondary storage rather than the OS SSD.

5. Docker Dynamic Data Migration

Container writes (Home Assistant, Nextcloud DB, n8n) default to /var/lib/docker on the OS drive.

- Move Docker runtime data to an external drive by editing /etc/docker/daemon.json:

JSON

{

"data-root": "/mnt/external_drive_1/docker-data"

}

- Restart Docker:

Bash

sudo systemctl restart docker

Step 4: Storage Allocation Profile (8GB RAM Pi)

Resource Layer	Allocation	Function
Active Applications	~4.5 GB	Home Assistant, n8n, Nextcloud, Samba,

		FileBrowser
Linux Page / Dirty Cache	~2.0 GB	Absorbs OS writes; batches flushes every 30 mins
tmpfs + log2ram	~0.75 GB	In-memory system logs, /tmp, /var/tmp
ZRAM Swap	~0.75 GB	Compressed RAM swap (disables SSD swap completely)
OS SSD Activity	Idle	Reads on demand; writes once per 30 mins or on shutdown
Network File Ingestion	Direct	Direct-to-target path on /mnt/drive_X